SIMATS ENGINEERING

ASSESSMENT TOOL 1

ANALYTICAL PROBLEM SOLVING

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1107

Course Outcome Covered: CO2: Design UML diagrams to represent system requirements and architecture. (BL3)

Assessment Tool Weightage: 40%

Questions: Any 4 (four problems attempted) **Total Marks:** 40

Problems Submitted

Problem No.	UML Diagram	System Modelled
1	Use Case Diagram	Smart Banking System
2	Class Diagram	Online Vehicle Rental System
3	Sequence Diagram	E-Commerce Shopping System
5	State Diagram	Smart Parking Management System

Student Details

Particular	Details
Name of the Student	M Nishanth
Register Number	192511154
Department / Year	B. Tech / 2nd Year
Date of Submission	16-10-2026

Annexure A: Code of Conduct

ANALYTICAL PROBLEM SOLVING

I ___**M Nishanth (192511154)** __ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ____M Nishanth_____

Date: _____16-10-2026_____

1. Introduction and Approach

Unified Modeling Language (UML) diagrams turn written requirements into a visual model that analysts, designers and clients can all read. Each diagram type answers a different question about a system: use case diagrams show what the system must do, class diagrams show its static structure, sequence diagrams show how objects interact over time, and state diagrams show how an object changes in response to events (Booch, Rumbaugh, & Jacobson, 2005; Object Management Group, 2017). Course Outcome CO2 asks the student to design such diagrams to represent system requirements and architecture.

Four of the six problems in Assessment Tool 1 have been solved, as required (4 problems × 10 marks = 40 marks). The four problems were chosen to cover the requirements view, the structural view and the behavioural view of a system.

1.1 Problems Selected

Problem	UML Diagram	View of the System	Marks
1	Use Case Diagram	Requirements (functional view)	10
2	Class Diagram	Structure (static view)	10
3	Sequence Diagram	Behaviour (interaction over time)	10
5	State Diagram	Behaviour (life cycle of an object)	10
	Total		**40**

1.2 Method Followed for Every Problem

- **Problem understanding:** the scenario is read closely, the stated requirements are listed, and assumptions are written down wherever the problem says the requirements are incomplete or unclear.
- **Method selection:** the UML diagram type is justified and its notation is explained.
- **Solution process:** tasks (a), (b) and (c) are solved in the order given, each in a table, and then used to construct the diagram in task (d).
- **Accuracy check:** the finished diagram is checked against the requirements and against the UML notation rules.
- **Interpretation:** what the diagram shows, its benefits, and its limitations or assumptions are explained.

1.3 Layout of this Document

Each complete diagram is placed on its own landscape page so that it can be read clearly. The tables that support it appear on the portrait pages before and after it. All diagrams follow UML 2.5.1 notation.

2. Problem 1: Use Case Diagram (Smart Banking System)

2.1 Problem Understanding

A Smart Banking System lets customers create accounts, transfer funds, pay bills and view their transaction history. Bank employees verify customer details, and administrators manage user accounts and banking services. The problem states that the requirements are only partially documented, so some supporting behaviour has to be inferred.

Stated requirements:

- R1: Customers can create accounts.
- R2: Customers can transfer funds.
- R3: Customers can pay bills.
- R4: Customers can view transaction history.
- R5: Bank employees verify customer details.
- R6: Administrators manage user accounts and banking services.

Assumptions made to complete the requirements:

- A1: Customers must be authenticated before a transfer, bill payment or history request.
- A2: Transfers to another bank need an extra one-time password (OTP) check.
- A3: A customer may optionally download a statement of the transaction history.
- A4: Billers and payment gateways are treated as part of the system, so they are not drawn as separate actors.

2.2 Method Selection

A **use case diagram** is the most suitable method because the problem is about requirements. It shows the goals of each user of the system and the functions the system offers to reach those goals, without describing how they are built (Jacobson, Christerson, Jonsson, & Övergaard, 1992; Cockburn, 2001). A class or sequence diagram would be too detailed at this stage. The notation used is summarised below.

Element	Notation	Meaning
Actor	Stick figure outside the boundary	A role that interacts with the system.

Element	Notation	Meaning
Use case	Oval inside the boundary	A function that gives value to an actor.
Association	Plain line	An actor takes part in a use case.
Include	Dashed arrow labelled «include», from base to included use case	Mandatory behaviour that is reused.
Extend	Dashed arrow labelled «extend», from extension to base use case	Optional or conditional behaviour.
Generalization	Solid line with hollow triangle at the parent	A specialised use case inherits from a general one.

2.3 Task (a): Actors Involved

Actor	Type	Role in the System
Customer	Primary	Creates an account, transfers funds, pays bills and views transaction history.
Bank Employee	Primary (staff)	Verifies the identity and details of customers when an account is created.
Administrator	Primary (staff)	Manages user accounts and banking services.

2.4 Task (b): Primary Use Cases

ID	Use Case	Actor	Type	Description
UC1	Create Account	Customer	Primary	Customer submits personal details to open a new account.
UC2	Transfer Funds	Customer	Primary	Customer moves money from an account to another account.
UC3	Pay Bills	Customer	Primary	Customer pays a utility or service bill from an account.
UC4	View Transaction History	Customer	Primary	Customer views past credits and debits.

ID	Use Case	Actor	Type	Description
UC5	Verify Customer Details	Bank Employee	Primary	Employee checks identity documents and details.
UC6	Manage User Accounts	Administrator	Primary	Administrator activates, updates, blocks or removes accounts.
UC7	Manage Banking Services	Administrator	Primary	Administrator configures services, limits and charges.
UC8	Authenticate User	Customer	Supporting	Login with credentials before using protected functions.
UC9	Transfer Within Bank	Customer	Supporting	Transfer between two accounts of the same bank.
UC10	Transfer to Other Bank	Customer	Supporting	Transfer to an account in another bank.
UC11	Verify OTP	Customer	Supporting	Extra one-time password check for external transfers.
UC12	Download Statement	Customer	Supporting	Optional download of a statement.

2.5 Task (c): Relationships Between Use Cases

From	Relationship	To	Justification
Create Account	«include»	Verify Customer Details	An account cannot be opened until an employee has verified the details, so this step is mandatory.
Transfer Funds	«include»	Authenticate User	Every transfer needs a verified customer.
Pay Bills	«include»	Authenticate User	The same login behaviour is defined once and reused.
View Transaction History	«include»	Authenticate User	Account data is shown only after login.
Verify OTP	«extend»	Transfer to Other Bank	Optional, conditional step added only for external transfers.

From	Relationship	To	Justification
Download Statement	«extend»	View Transaction History	Optional step used only when the customer asks for it.
Transfer Within Bank	Generalization	Transfer Funds	Specialised transfer between two accounts of this bank.
Transfer to Other Bank	Generalization	Transfer Funds	Specialised transfer with extra checks.

2.6 Task (d): Complete Use Case Diagram

The complete Use Case Diagram, built from the actors, use cases and relationships above, is given as Figure 1 on the next page.

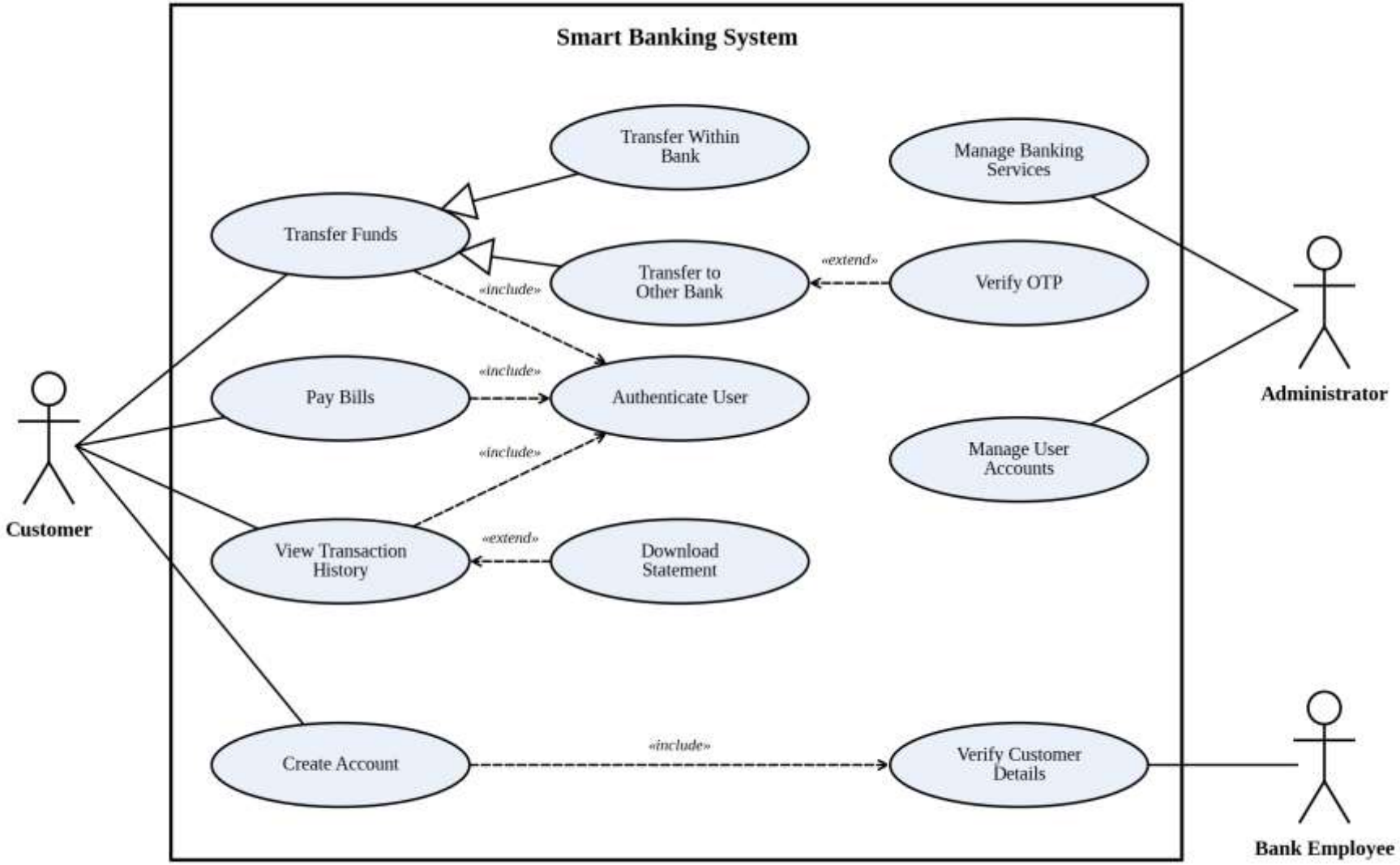


Figure 1: Use Case Diagram of the Smart Banking System

2.7 Requirement Traceability

Requirement	Realised by
R1: Customers create accounts	UC1 Create Account
R2: Customers transfer funds	UC2 Transfer Funds (specialised by UC9 and UC10)
R3: Customers pay bills	UC3 Pay Bills
R4: Customers view transaction history	UC4 View Transaction History (extended by UC12)
R5: Employees verify customer details	UC5 Verify Customer Details
R6: Administrators manage user accounts and services	UC6 Manage User Accounts and UC7 Manage Banking Services

2.8 Accuracy Check of the Diagram

Check	Result
Every stated requirement (R1 to R6) is covered by at least one use case.	Satisfied
Every actor is linked to at least one use case.	Satisfied
All use cases lie inside the system boundary and all actors lie outside it.	Satisfied
«include» arrows run from the base use case to the included use case.	Satisfied
«extend» arrows run from the extending use case to the base use case.	Satisfied
Generalization arrows (hollow triangle) point to the parent use case.	Satisfied

2.9 Interpretation of Results

- **Customers have four main goals** (create account, transfer funds, pay bills, view history). All four are directly linked to the Customer actor, so the system is customer-centred.
- **Staff roles are separated.** Employees only verify details, while administrators only manage accounts and services. This separation supports role-based access control and limits the damage of misuse.

- **«include» shows reuse and security.** Authenticate User is written once and used by three use cases, so a change to the login rules is made in one place.
- **«extend» keeps optional behaviour out of the main flow.** OTP verification and statement download only happen when they are needed, so the basic use cases stay simple.
- **Limitations.** Assumptions A1 to A4 should be confirmed with bank stakeholders, because the requirements were only partially documented. The diagram also shows what the system does, not how, so class and sequence diagrams are the next step.

3. Problem 2: Class Diagram (Online Vehicle Rental System)

3.1 Problem Understanding

The Online Vehicle Rental System has five named entities: Vehicle, Customer, Reservation, Payment and RentalAgent. The problem says that the relationships between them are not clearly defined, so they have to be worked out from the way a rental business works. The following business rules were derived and used to decide every relationship and multiplicity.

- A customer can make many reservations over time, and each reservation is made by exactly one customer.
- Each reservation is for exactly one vehicle, and a vehicle can have many reservations over time (on different dates).
- Each reservation is processed by exactly one rental agent, and an agent processes many reservations.
- A payment exists only for a reservation (for example a deposit and a final payment), and is removed if the reservation is removed.
- Cars, bikes and trucks share common vehicle data but have their own details and rent calculation.
- Vehicles are kept at rental branches, but a vehicle can be moved from one branch to another.

The class Branch and the subclasses Car, Bike and Truck are additions to the five named entities. They are needed to show aggregation and inheritance, which the problem asks for "where applicable".

3.2 Method Selection

A **class diagram** is selected because it shows the static structure of a system: the classes, their attributes and methods, and how they are related (Fowler, 2003). It is the main design model of object-oriented analysis and design and can be turned into code directly. The relationship types used are listed below.

Relationship	Notation	Used When
Association	Plain line with name and multiplicity	Objects of two classes are linked and can exist independently.

Relationship	Notation	Used When
Aggregation	Hollow diamond at the whole	A whole holds parts, but the parts can exist without it.
Composition	Filled diamond at the whole	The part cannot exist without the whole and is deleted with it.
Inheritance	Hollow triangle at the parent	A subclass is a specialised kind of the parent class.

3.3 Task (a): Classes, Attributes and Methods

Class	Attributes	Methods
Vehicle (abstract)	- vehicleId: String - registrationNo: String - dailyRate: double - status: String	+ isAvailable(): boolean + calculateRent(days): double
Car	- seats: int - fuelType: String	+ calculateRent(days): double
Bike	- engineCC: int - hasHelmet: boolean	+ calculateRent(days): double
Truck	- loadCapacity: double - permitNo: String	+ calculateRent(days): double
Customer	- customerId: String - name: String - phone: String - licenseNo: String	+ register(): void + makeReservation(): Reservation + cancelReservation(): void
Reservation	- reservationId: String - startDate: Date - endDate: Date - totalAmount: double - status: String	+ confirm(): void + cancel(): void + calculateTotal(): double
Payment	- paymentId: String - amount: double - status: String	+ processPayment(): boolean + refund(): void

Class	Attributes	Methods
RentalAgent	- agentId: String - name: String - phone: String	+ verifyLicense(): boolean + approveReservation(): void + handOverVehicle(): void
Branch	- branchId: String - city: String	+ getAvailableVehicles(): List

Attributes are private (-) to support encapsulation, and methods are public (+) so that other classes can use them.

3.4 Task (b): Relationships Between Classes

From	Relationship	To	Meaning
Customer	Association (makes)	Reservation	A customer makes reservations.
RentalAgent	Association (processes)	Reservation	An agent processes reservations.
Reservation	Association (reserves)	Vehicle	A reservation books one vehicle.
Reservation	Composition (includes)	Payment	Payments belong to a reservation and are deleted with it.
Branch	Aggregation (stocks)	Vehicle	A branch holds vehicles that can be moved to another branch.
Vehicle	Inheritance	Car, Bike, Truck	Each vehicle type is a specialised Vehicle.

3.5 Task (c): Multiplicity Between Classes

Association	At First Class	At Second Class	Justification
Customer – Reservation	1	0..*	A customer may have no reservation or many; a reservation has exactly one customer.
RentalAgent – Reservation	1	0..*	Each reservation is handled by one agent; an agent handles many.

Association	At First Class	At Second Class	Justification
Vehicle – Reservation	1	0..*	A vehicle is booked many times over time; each reservation is for one vehicle.
Reservation – Payment	1	1..*	A reservation has at least one payment (for example deposit and balance).
Branch – Vehicle	1	0..*	A branch stocks zero or more vehicles; each vehicle is based at one branch.

3.6 Task (d): Detailed Class Diagram

The detailed Class Diagram showing all classes, relationships and multiplicities is given as Figure 2 on the next page.

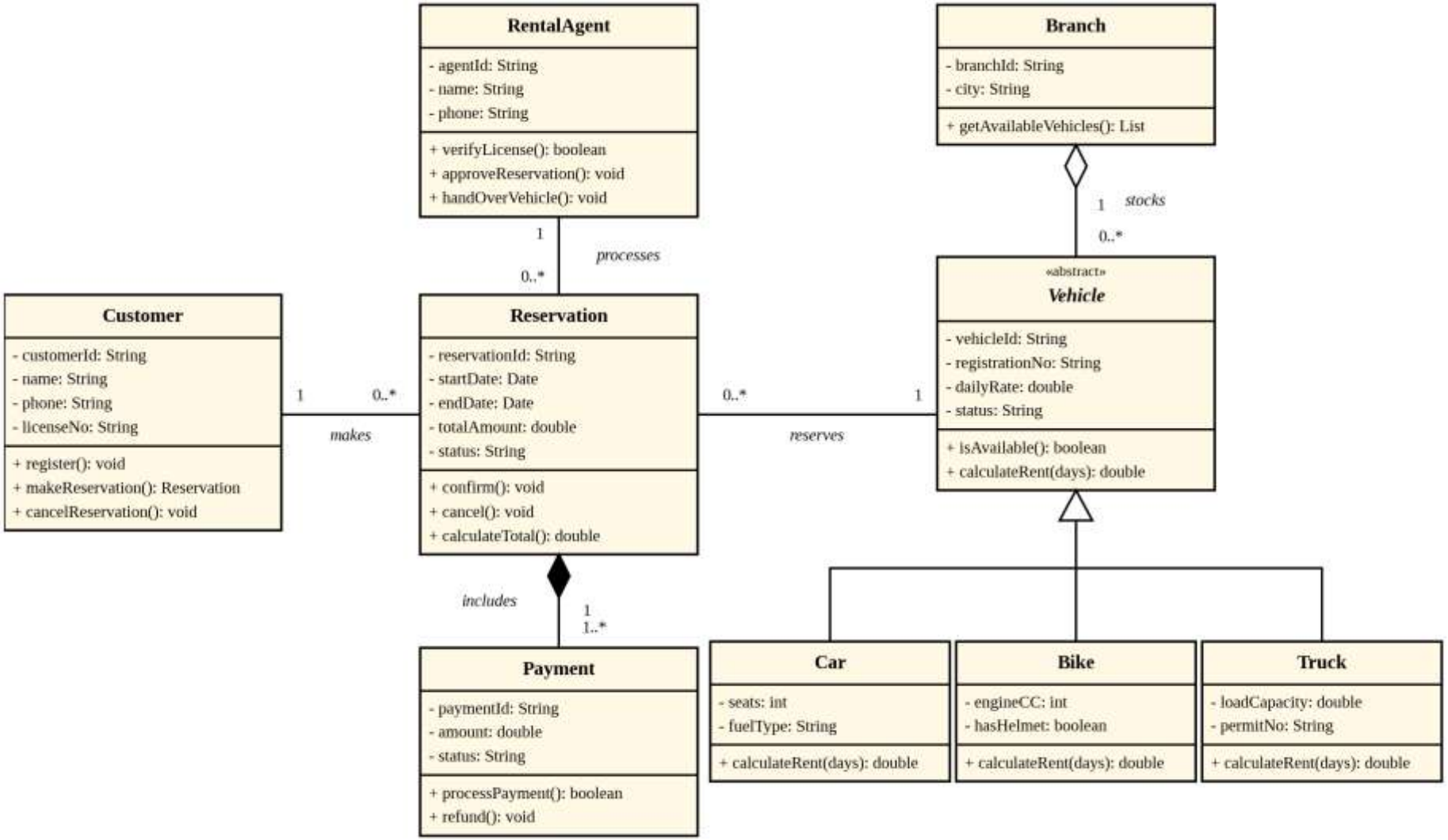


Figure 2: Class Diagram of the Online Vehicle Rental System

3.7 Accuracy Check of the Diagram

Check	Result
All five named classes (Vehicle, Customer, Reservation, Payment, RentalAgent) are present.	Satisfied
Every association has a name and a multiplicity at both ends.	Satisfied
The composition diamond is filled and sits at the whole (Reservation).	Satisfied
The aggregation diamond is hollow and sits at the whole (Branch).	Satisfied
The inheritance triangle points to the parent (Vehicle), which is marked abstract.	Satisfied
Each subclass overrides calculateRent(days), so the rule differs by vehicle type.	Satisfied

3.8 Interpretation of Results

- **Reservation is the central class.** It links the customer, the vehicle, the agent and the payments, which matches how a rental actually works.
- **Composition protects data integrity.** Because payments are owned by a reservation, no orphan payments remain when a reservation is deleted. A vehicle is not owned by a reservation or a branch, so those links are association and aggregation.
- **Inheritance and polymorphism reduce duplication.** Common vehicle data is stored once in Vehicle, and calculateRent(days) gives a different result for a car, a bike or a truck through one common call.
- **Encapsulation.** Private attributes can only be changed through methods, which keeps objects in a valid state.
- **Limitations.** Branch and the vehicle subclasses are design decisions added to the five named classes, and the multiplicities depend on the business rules in Section 3.1. They should be confirmed with the rental company before coding.

4. Problem 3: Sequence Diagram (E-Commerce Shopping System)

4.1 Problem Understanding

The E-Commerce Shopping System lets a customer browse products, add items to a cart, place an order, make a payment and receive an order confirmation. The problem says the order of interactions between the system components is unclear, so the components and their order of messages have to be defined.

The flow was divided into five phases: **browse** products, **add to cart**, **place order**, **pay**, and **confirm or fail**. A payment can succeed or fail, so the last phase has two paths. The following assumptions were made.

- The customer works through a single front-end object, the ShoppingPortal.
- The OrderService controls the order and uses the ProductCatalog to check stock.
- Card details are passed to a separate PaymentService, so the shop itself does not process payments.
- Order confirmation is sent through a NotificationService without making the customer wait.

4.2 Method Selection

A **sequence diagram** is selected because the main question is the order of messages between objects. It shows each object as a lifeline and each message as an arrow placed from top to bottom in time order, so the flow can be read like a script (Fowler, 2003). A communication diagram could show the same messages, but it emphasises object links and not time order, so it is less suitable here. The notation is summarised below.

Element	Notation	Meaning
Lifeline	Box with a dashed vertical line	The life of an object during the interaction.
Activation bar	Thin rectangle on the lifeline	The period in which the object is busy.
Synchronous message	Solid line, filled arrowhead	The sender waits for a reply.
Return message	Dashed line, open arrowhead	The reply to an earlier call.

Element	Notation	Meaning
Asynchronous message	Solid line, open arrowhead	The sender continues without waiting.
Self message	Arrow that loops back to the same lifeline	An object calls its own operation.
Alt fragment	Frame with a dashed divider	Alternative paths chosen by a condition.

4.3 Task (a): Objects Participating in the Interaction

Object	Role	Responsibility
Customer	Actor	Starts every interaction: browses, adds items, orders and pays.
:ShoppingPortal	Front end	Receives the customer's requests and shows the results.
:ProductCatalog	Entity / service	Provides the product list and checks stock.
:ShoppingCart	Entity	Holds the items chosen by the customer.
:OrderService	Controller	Creates the order, handles payment results and confirms the order.
:PaymentService	External service	Authorizes the payment and reports success or failure.
:NotificationService	Service	Sends the order confirmation to the customer.

4.4 Task (b): Message Flow Between Objects

No.	From	To	Message	Type
1	Customer	ShoppingPortal	browseProducts()	Synchronous
2	ShoppingPortal	ProductCatalog	getProducts()	Synchronous
3	ProductCatalog	ShoppingPortal	productList	Return
4	ShoppingPortal	Customer	displayProducts()	Return
5	Customer	ShoppingPortal	addToCart(id, qty)	Synchronous

No.	From	To	Message	Type
6	ShoppingPortal	ShoppingCart	addItem(id, qty)	Synchronous
7	ShoppingCart	ShoppingPortal	cartUpdated	Return
8	Customer	ShoppingPortal	placeOrder()	Synchronous
9	ShoppingPortal	OrderService	createOrder(cart)	Synchronous
10	OrderService	ProductCatalog	checkStock(items)	Synchronous
11	ProductCatalog	OrderService	stockAvailable	Return
12	OrderService	ShoppingPortal	orderCreated	Return
13	Customer	ShoppingPortal	makePayment(details)	Synchronous
14	ShoppingPortal	OrderService	processPayment(orderId)	Synchronous
15	OrderService	PaymentService	authorize(amount)	Synchronous
16	PaymentService	OrderService	paymentStatus	Return
17	OrderService	OrderService	confirmOrder()	Self (if successful)
18	OrderService	NotificationService	sendConfirmation(orderId)	Asynchronous (if successful)
19	OrderService	ShoppingPortal	orderConfirmed	Return (if successful)
20	ShoppingPortal	Customer	displayConfirmation()	Return (if successful)
21	OrderService	ShoppingPortal	paymentFailed	Return (else)
22	ShoppingPortal	Customer	showErrorMessage()	Return (else)

4.5 Task (c): Activation Bars and Lifelines

Every object has a lifeline (the dashed vertical line). The table shows when each object is active, which is drawn as an activation bar on its lifeline.

Object	Active During Messages	Reason
Customer	Lifeline only	The customer is an actor outside the system, so no activation bar is drawn.

Object	Active During Messages	Reason
:ShoppingPortal	1–4, 5–7, 8–12, 13–22	Active from receiving each customer request until it has replied.
:ProductCatalog	2–3 and 10–11	Active while returning products and while checking stock.
:ShoppingCart	6–7	Active while adding an item and confirming the update.
:OrderService	9–12 and 14–21	Active while creating the order and while handling the payment result.
:PaymentService	15–16	Active while authorizing the payment.
:NotificationService	18	Active briefly while sending the confirmation.

4.6 Task (d): Complete Sequence Diagram

The complete Sequence Diagram showing lifelines, activation bars and the full message flow, including the payment success and failure paths, is given as Figure 3 on the next page.

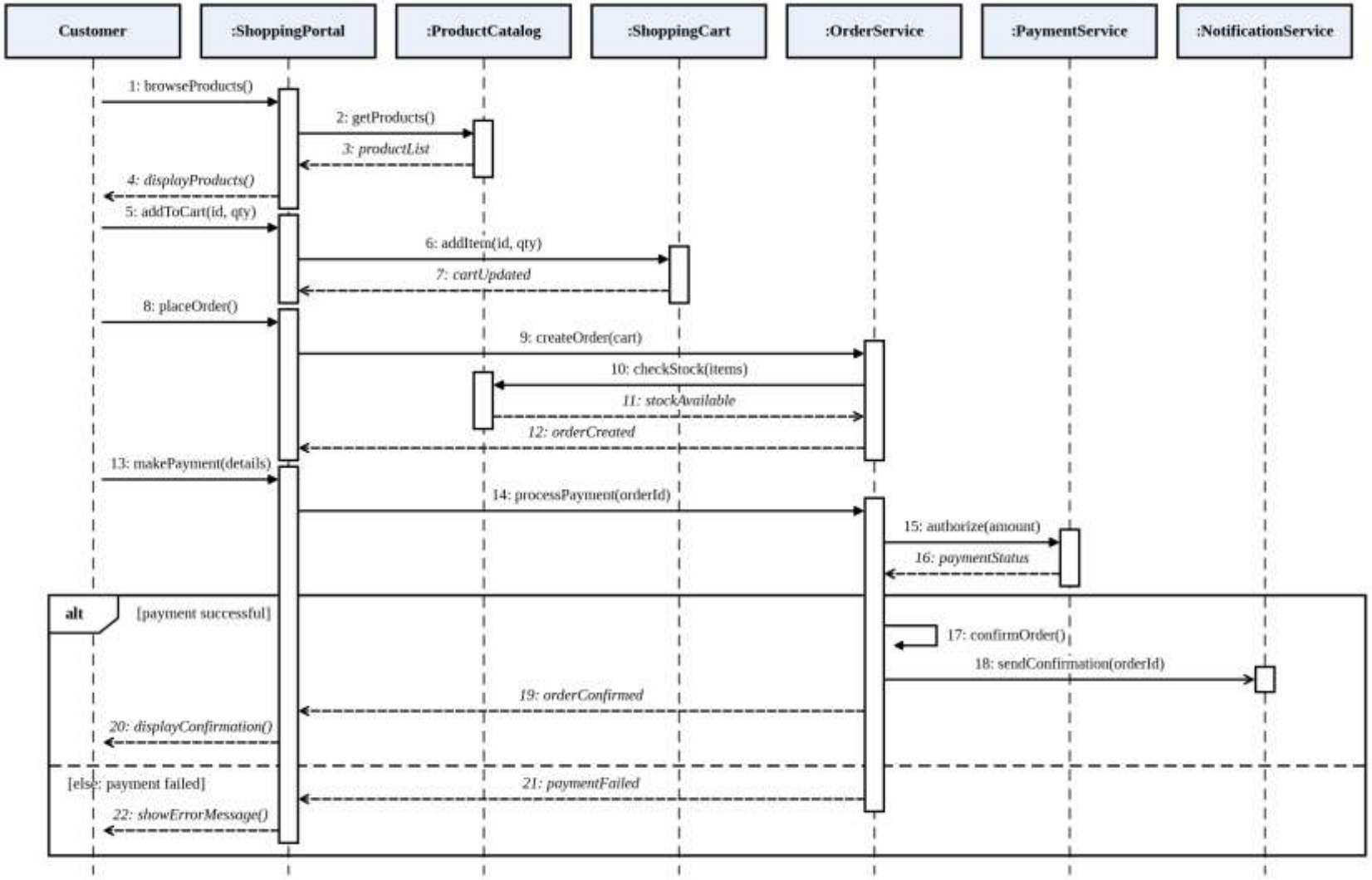


Figure 3: Sequence Diagram of the E-Commerce Shopping System

4.7 Accuracy Check of the Diagram

Check	Result
All seven participating objects have a lifeline.	Satisfied
Messages are numbered 1 to 22 in strict top-to-bottom time order.	Satisfied
Every synchronous call (solid, filled arrowhead) has a matching return (dashed arrow).	Satisfied
Activation bars start at the call received and end at the reply sent.	Satisfied
The alt fragment shows both outcomes of the payment, with a guard on each path.	Satisfied
The notification is asynchronous, so the customer is not kept waiting.	Satisfied

4.8 Interpretation of Results

- **The OrderService is the coordinator.** It talks to the catalog, the payment service and the notification service, so the portal stays simple and the business rules are kept in one place.
- **Payment is isolated.** Only the PaymentService authorizes the payment, which keeps sensitive card handling away from the rest of the shop.
- **The alt fragment covers the exception path.** A failed payment ends with a clear error message to the customer, so the design does not assume that everything succeeds.
- **Messages become methods.** Each message is an operation of the receiving class (for example, checkStock(items) belongs to ProductCatalog), so this diagram feeds directly into the class diagram.
- **Limitations.** Real systems also need timeouts, retry rules and stock reservation. These can be added as further fragments when the requirements are extended.

5. Problem 5: State Diagram (Smart Parking Management System)

5.1 Problem Understanding

A Smart Parking Management System works through the states Parking Slot Available, Vehicle Entered, Parking Allocated, Payment Pending, Payment Completed and Vehicle Exited. The behaviour of the system during parking operations must be modelled.

The diagram models the life cycle of one parking slot as a vehicle uses it: the slot is free, a vehicle arrives, a slot is allocated, the vehicle stays, a payment is made, and the vehicle leaves, after which the slot is free again. The following assumptions were made.

- A vehicle that enters but is not allocated a slot before a time limit is cancelled and the slot becomes available again.
- A failed payment can be retried up to three times before help is needed.
- A normal slot cycles back to Available after each vehicle. The final state is reached only when the slot is taken out of service (for example, for closure or maintenance).

5.2 Method Selection

A **state diagram** (state machine diagram) is selected because the problem is about how one thing changes state in response to events over its life. It shows each state, the events that cause a change, and the actions that follow (Harel, 1987; Object Management Group, 2017). An activity diagram would show the workflow steps, but not clearly the condition of the system between steps. The notation is summarised below.

Element	Notation	Meaning
State	Rounded rectangle	A condition in which the system stays until an event occurs.
Initial state	Filled circle	Where the behaviour begins.
Final state	Circle with a filled centre	Where the behaviour ends.
Transition	Arrow labelled event [guard] / action	A change of state caused by an event.
Entry action	entry / action inside a state	Action performed when the state is entered.

5.3 Task (a): States of the System

State	Meaning	Entry Action
Initial state	System starts and the slot is put in service.	None
Slot Available	The slot is free and shown as free to drivers.	showSlotFree()
Vehicle Entered	A vehicle has been detected at the entrance.	captureNumberPlate()
Parking Allocated	A slot has been assigned and a ticket issued.	assignSlot(), printTicket()
Payment Pending	The vehicle wants to leave and the fee is waiting to be paid.	calculateFee()
Payment Completed	The fee has been paid successfully.	issueReceipt()
Vehicle Exited	The vehicle has left through the exit gate.	openExitGate()
Final state	The slot is taken out of service.	None

5.4 Task (b): Events Triggering State Transitions

No.	From	Event [Guard] / Action	To
1	Initial state	(system start)	Slot Available
2	Slot Available	vehicleDetected	Vehicle Entered
3	Vehicle Entered	slotAssigned [slot free]	Parking Allocated
4	Vehicle Entered	entryCancelled [timeout]	Slot Available
5	Parking Allocated	exitRequested / calculateFee()	Payment Pending
6	Payment Pending	paymentSuccess	Payment Completed
7	Payment Pending	paymentFailed [retries < 3] / retryPayment()	Payment Pending
8	Payment Completed	gateOpened	Vehicle Exited
9	Vehicle Exited	slotReleased / updateAvailability()	Slot Available
10	Slot Available	slotDecommissioned	Final state

5.5 Task (c): Initial and Final States

The **initial state** is entered when the system starts, and it leads directly to Slot Available. The **final state** is reached from Slot Available when the slot is decommissioned. The final state is not placed after Vehicle Exited, because a parking slot is reused many times, and a slot that is returned to service after each vehicle must not end its life after the first one.

5.6 Task (d): State Transition Diagram

The complete State Transition Diagram, with all states, events, guards, entry actions, and the initial and final states, is given as Figure 4 on the next page.

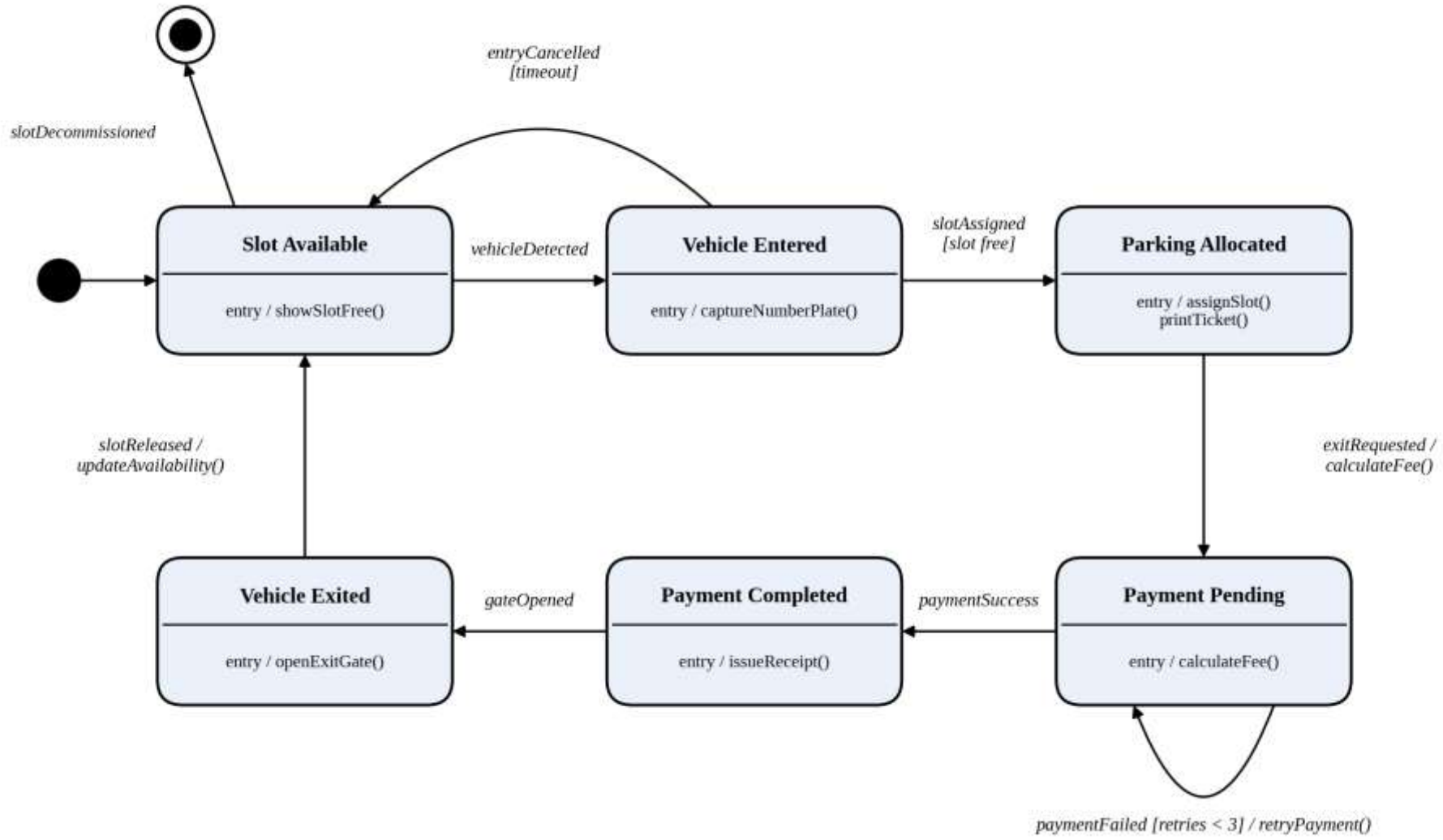


Figure 4: State Diagram of the Smart Parking Management System

5.7 Accuracy Check of the Diagram

Check	Result
All six states given in the problem are present.	Satisfied
There is exactly one initial state and it has one outgoing transition.	Satisfied
The final state has no outgoing transitions and is reachable.	Satisfied
Every transition has an event; guards and actions are shown where needed.	Satisfied
Every state can be reached from the initial state and, except the final state, can be left.	Satisfied
The failed-payment loop is a self-transition that is limited by the guard [retries < 3].	Satisfied

5.8 Interpretation of Results

- **The main flow is a cycle.** A slot moves from Available to Vehicle Exited and then returns to Available, which shows that slots are reused and that the system does not stop after one vehicle.
- **Exceptions are handled.** An entry that times out returns to Available, and a failed payment is retried, so the design copes with real-world problems and not only the ideal path.
- **Payment gates the exit.** The exit gate opens only after Payment Completed, so a vehicle cannot leave without paying.
- **Entry actions map to operations.** Actions such as assignSlot() and calculateFee() become methods of the parking controller, which links this diagram to the class diagram.
- **Limitations.** The diagram models one slot. Overstay penalties, reserved slots and fault states are not shown and can be added as extra states if required.

6. Conclusion

Four analytical problems were solved by constructing the UML diagram that suits each one. The table below summarises the result.

Problem	Diagram	View	Main Outcome
1	Use Case	Requirements	3 actors and 12 use cases with include, extend and generalization relationships.
2	Class	Structure	9 classes with attributes, methods, relationships and multiplicities.
3	Sequence	Interaction	7 objects and 22 messages, with activation bars and a payment alt fragment.
5	State	Behaviour	6 states, 10 transitions, initial and final states and a retry guard.

Together the four diagrams describe one system from several sides: what it must do, what it is made of, how its parts interact, and how it changes over time. Each diagram was checked against the stated requirements and the UML notation rules, and each shows where the assumptions should be confirmed with stakeholders. This meets Course Outcome CO2, which is to design UML diagrams that represent system requirements and architecture.

7. Alignment with the Assessment Rubric

Rubric Criterion	Weightage	Where it is Addressed
Problem Understanding	20%	The Problem Understanding section of each problem lists requirements, rules and assumptions.
Method Selection	20%	The Method Selection section justifies each diagram and explains its notation.
Solution Process	25%	Tasks (a), (b), (c) and (d) are solved in the order given, each with a table, before the diagram.
Accuracy of Calculation	20%	UML modelling has no numerical calculation, so accuracy is shown by the Accuracy Check tables (notation, multiplicity, messages and transitions).
Interpretation of Results	15%	The Interpretation of Results section of each problem explains the meaning, benefits and limitations.

References

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language user guide* (2nd ed.). Addison-Wesley.

Cockburn, A. (2001). *Writing effective use cases*. Addison-Wesley.

Fowler, M. (2003). *UML distilled: A brief guide to the standard object modeling language* (3rd ed.). Addison-Wesley.

Harel, D. (1987). Statecharts: A visual formalism for complex systems. *Science of Computer Programming, 8*(3), 231–274.

Jacobson, I., Christerson, M., Jonsson, P., & Övergaard, G. (1992). *Object-oriented software engineering: A use case driven approach*. Addison-Wesley.

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3rd ed.). Prentice Hall.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1